National Tsing Hua University

Error Correcting Codes II

Final Project Report

Rate 1/2-LDPC Code Simulation

Njini Nathan Fofeyin

112064422

恩哲倪

November 26, 2025

Abstract

This report presents the simulation and performance evaluation of a WiMAX-style Low-Density Parity-Check (LDPC) code with a rate of 1/2 and a codeword length of 2304 bits. The encoder is implemented using a double-diagonal parity construction with an expansion factor $Z = 96$, based on a standardized 12×24 base matrix. Decoding is performed using the log-domain Sum-Product Algorithm (log-SPA) with the $\phi(\cdot)$ function approximation for improved numerical stability. The minsum algorithm is laso implemented. The simulations are conducted over an additive white Gaussian noise (AWGN) channel using binary phase-shift keying (BPSK) modulation. Bit error rate (BER) performance is analyzed across a range of signal-to-noise ratios (SNRs), demonstrating the error-correcting capability of the code. The results validate the expected coding gain and confirm the effectiveness of the log-SPA decoder implementation and minsum algorithm.

Contents

Abstract **3**

1. Introduction **4**

2. Encoder **6**
2.1. Code Construction 6
2.2. Dual Diagonal Encoding 7

3. Channels **9**
3.1. BPSK over BI-AWGNC 9

4. LDPC Decoding **10**
4.1. Log Sum Product Algorithm 10
4.2. Minsum Algorithm 12

5. Results and Discussion **14**
5.1. BER Performance 14
5.2. BER Performance of Minsum vs Log SPA 15
5.3. Complexity analysis 16

6. Conclusion and Future Work **17**

A. Monte Carlo Methods and Useful Statistics **18**

1. Introduction

LDPC Codes were first introduced by Robert Gallager in 1960. Since then, they have been adopted into several technology standards. In this project, an LDPC code simulation is completed. The code is from the WiMaX (IEEE 802.16e) standard, and it consists of several properties that makes it suitable and efficient for error correction and reliable channel coding [6].

In this project, we primarily consider the log domain sum product algorithm (hereafter referred to as log SPA) to study the bit error rate (BER) performance for a rate 1/2 (m, n)-LDPC code where $m = 1152$ and $n = 2304$. As is common for LDPC codes in standards such as 5G NR, WiFi (IEEE 802.11n), satellite communication and WiMaX, our code design has a special structure, completely characterized by $(\mathbf{B}, Z, \mathbf{I}_Z, \text{rate})$, where $\mathbf{B}$ is the base matrix (also known as protograph), the expansion factor Z, $\mathbf{I}_Z$ is a $Z \times Z$ identity matrix, with rate representing the fraction of message bits to the entire codeword.

These parameters can be used to directly construct the parity check matrix $\mathbf{H}$ of the code which can be used primarily for decoding codewords. In our given code, the $\mathbf{H}$ matrix has a special double diagonal structure. In the next section, we will discuss the implications of this structure for encoding, eliminating the need for a generator matrix $\mathbf{G}$ for the encoding process.

In this project, we realize a complete end-to-end simulation of 1152×2304 LDPC code of rate 1/2 was simulated and the BER performance over an AWGN channel using both the log SPA algorithms and minsum algorithms. We perform this simulation using monte carlo method by randomly generating a large number of random bits and performing the simulation until we have a converged BER rate. We utilize a fairly large number of code words (`max_num_rounds <= 150000`). We significantly reduce the amount of simulation time required to perform the simulation by checking for convergence of the BER to the mean of the previous `100` BER values. We deem this value appropriate for this simulation, and ensure that at least a confidence interval of 0.95 is achieved. Elaborate details are provided in section 5.

Each of 3 blocks are carefully modelled and implemented to realize the simulation. These include the encoder, channel and decoder as shown in figure 1.1. Each of these play the following roles:

Figure 1.1.: End-to-end simulation framework.

1. **Encoder:** Maps input message bits to a codeword using the LDPC parity check matrix $\mathbf{H}$. This is achieved using the dual diagonal encoding technique.

2. **Channel:** Introduces noise or distortions modeled by a specific channel model (e.g., AWGN, Rayleigh, BSC etc). In our case, we utilize a BI-AWGNC $\mathcal{N}(0, \sigma^2)$, where sigma is the channel's noise variance.

3. **Decoder:** Estimates transmitted codeword bits from noisy observations using a decoding algorithm such as log SPA, or minsum. These are generally referred to as belief propagation algorithms.

It is worth noting that from the codeword, the decoded message bits can then be directly extracted depending on the rate and encoding technique. In our case, we obtain the message by directly grabbing the first half bits from the decoded codeword due to systematic encoding and our code rate $r = \frac{1}{2}$. This is also possible due to no interleaving in the encoding step. The number of message bits is determined by the code rate.

We establish that the code is capable of achieving low error rates of the order $\sim 10^{-5}$ to $\sim 10^{-7}$ for high SNR. These statistical estimates are obtained by simulating a large number of bits until a converged BER value is obtained.

2. Encoder

2.1. Code Construction

The parity check matrix **H** completely characterizes an LDPC code. In standards, this is usually constructed from a so-called base matrix (protograph). In our case, the base matrix **B** is given as follows

-1	94	73	-1	-1	-1	-1	-1	55	83	-1	-1	7	0	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	27	-1	-1	-1	22	79	9	-1	-1	-1	12	-1	0	0	-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	-1	-1	24	22	81	-1	33	-1	-1	-1	0	-1	-1	0	0	-1	-1	-1	-1	-1	-1	-1	-1
61	-1	47	-1	-1	-1	-1	-1	65	25	-1	-1	-1	-1	-1	0	0	-1	-1	-1	-1	-1	-1	-1
-1	-1	39	-1	-1	-1	84	-1	-1	41	72	-1	-1	-1	-1	-1	0	0	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	46	40	-1	82	-1	-1	-1	79	0	-1	-1	-1	-1	0	0	-1	-1	-1	-1	-1
-1	-1	95	53	-1	-1	-1	-1	-1	14	18	-1	-1	-1	-1	-1	-1	-1	0	0	-1	-1	-1	-1
-1	11	73	-1	-1	-1	2	-1	-1	47	-1	-1	-1	-1	-1	-1	-1	-1	-1	0	0	-1	-1	-1
12	-1	-1	-1	83	24	-1	43	-1	-1	-1	51	-1	-1	-1	-1	-1	-1	-1	-1	0	0	-1	-1
-1	-1	-1	-1	-1	94	-1	59	-1	-1	70	72	-1	-1	-1	-1	-1	-1	-1	-1	-1	0	0	-1
-1	-1	7	65	-1	-1	-1	-1	39	49	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	0	0
43	-1	-1	-1	-1	66	-1	41	-1	-1	-1	26	7	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	0

Figure 2.1.: Base Matrix, **B**

The base matrix indicates shifting coefficients such that the resulting parity matrix is sparse and irregular LDPC code. The resulting code is a quasi-cyclic LDPC code since each identity matrix block $\mathbf{I}_{96\times 96}$ has shifted columns. The shifting mechanism is designed to make the matrix sparse and to generate seemingly random codes towards achieving channel capacity. It is also designed to avoid short cycles in the Tanner graph, which can affect decoding performance, especially for BEC channels (although not a major concern for this project).

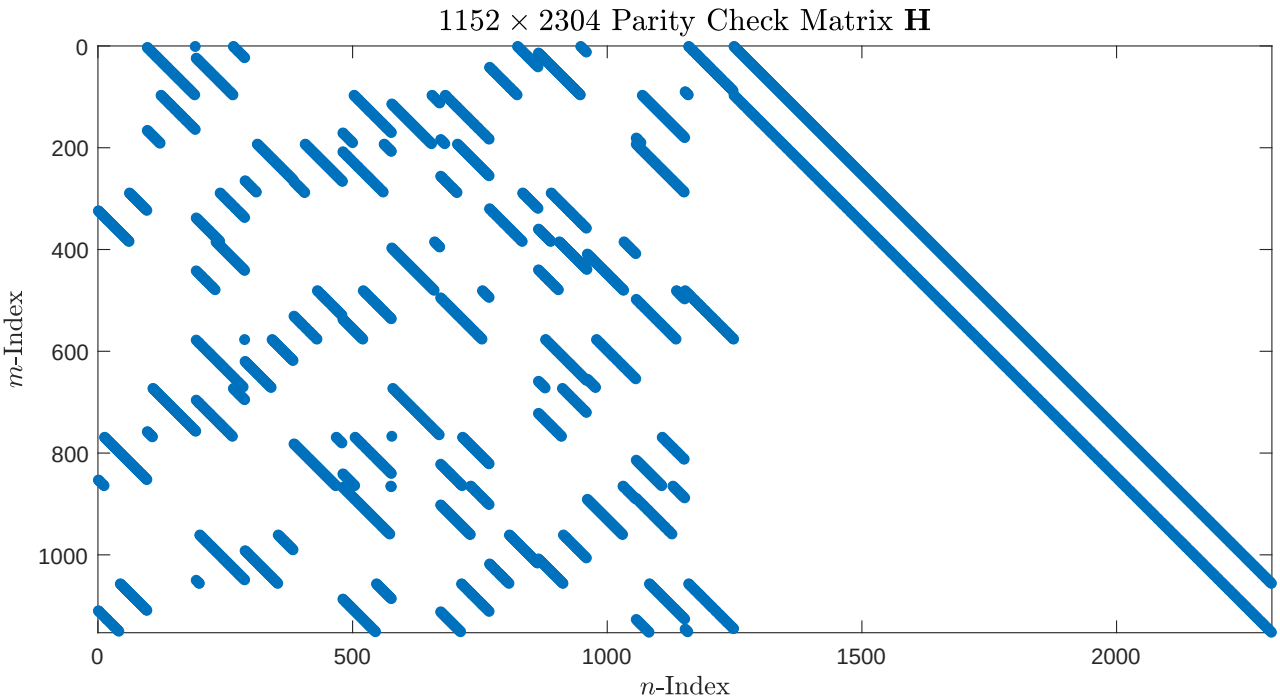

Figure 2.2.: Sparsity visualization of the parity-check matrix $\mathbf{H}$.

It is important to notice the $\mathbf{H}$ matrix is large but sparse. Hence, we must devise smart ways to work with it. It is common to implement it as an `.alist` file. In case however, we simply utilize a text file and save the entries with "1"'s to the text file. This is done both check-node-wise (rows) and variable-node-wise (columns). In the encoding process, we only utilize the check-node-based H matrix in the file `positions_cn_vn.txt`. The encoding process is elaborated in the following section.

2.2. Dual Diagonal Encoding

It is a general principle error correction coding to favour systematic encoding, moreso when it can be done efficiently. The key idea in our LDPC encoding is to calculate the parity bits $\mathbf{p} \in GF(2^m)$, $m = 1152$, which are be appended to the message bits to form a codeword. This is known as systematic encoding. Although our $\mathbf{H}$ is not in the typical systematic form, it has a special dual diagonal structure, as observed from figure 2.2. Here, we prove this dual diagonal structure from columns $1249 \rightarrow 2304$ across all 1152 rows, can enable systematic encoding via efficient calculation of the parity bits [2]. To prove this, we proceed with the following proof:

Let $\mathbf{u} = [\mathbf{u}_1, \ldots, \mathbf{u}_{12}]$ be a message consisting of 1152 bits. We wish to compute the systematic codeword $\mathbf{c} = [\mathbf{u} \mid \mathbf{p}]$, where $\mathbf{p} = [\mathbf{p}_1, \ldots, \mathbf{p}_{12}]$ is the vector of parity bits.

Setup

Our systematic LDPC code with parity-check matrix $\mathbf{H} = [\mathbf{H}_m \mid \mathbf{H}_p]$ where $\mathbf{H}_m$ is $m \times m$ and $\mathbf{H}_p$ is $m \times m$) with dual-diagonal structure as shown in figures 2.1 and 2.2.

For message array $\mathbf{u}$, find parity array $\mathbf{p}$ such that codeword $\mathbf{c} = [\mathbf{u} \mid \mathbf{p}]$ satisfies $\mathbf{H}\mathbf{c}^{\mathrm{T}} = \mathbf{0}$.

Algorithm and Proof

- **Step 1: Syndrome Calculation**.

$$\mathbf{s} = \mathbf{H}_m \mathbf{u}^{\mathrm{T}} \tag{2.1}$$

- **Step 2: First Parity Block**. Using block summation (XOR) over circulant structure:

$$\mathbf{p}_0[j] = \bigoplus_{i=0}^{B-1} \mathbf{s}[iZ + j], \quad j = 0, \ldots, Z-1 \tag{2.2}$$

  where $B = 12$ blocks and $Z = 96$ block size.

- **Step 3: Sequential Parity Calculation**. For remaining parity bits p_k where $k \geq Z$:

$$p_k = \bigoplus_{j:H_{r,j}=1, j<k+n-m} c_j \tag{2.3}$$

  where r is the row corresponding to parity bit p_k.

Correctness

The algorithm produces valid codewords since for any parity check equation i:

$$\sum_{j=0}^{k-1} H_{i,j} u_j + \sum_{j=k}^{n-1} H_{i,j} p_{j-k} = s_i + \sum_{j=k}^{n-1} H_{i,j} p_{j-k} \tag{2.4}$$

By construction:

- Block summation ensures first Z equations are satisfied
- Sequential calculation ensures remaining equations are satisfied

Therefore: $\mathbf{H}\mathbf{c}^T = \mathbf{0}$ in $GF(2)$. $\square$

The algorithm achieves $O(n \cdot d_c)$ complexity where d_c is average check node degree, avoiding matrix inversion while maintaining systematic encoding. More detail on complexity is provided in section 5.3.

3. Channels

3.1. BPSK over BI-AWGNC

Modulation schemes convert binary sequence sets to channel-ready. In our simulation, we only implement BPSK, which simply maps input bits $u_j \in \{0, 1\}$ to $x_j \in \{\pm 1\}$. Several other modulation schemes also exist, BPSK is foundational and regardless of the order of modulation as is the case for QPSK, 16-QAM etc. These would map to different signals for transmission. BPSK is convenient for our simulation since it provides maximum phase separation of 2π, improving channel reliability and easily hard-decision decoding.

Using inputs from BPSK, the BI-AWGNC channel is modelled as

$$y_j = x + n_j,\ n_j \sim \mathcal{N}(0, \sigma^2) \text{ and } x = \{\pm 1\}\ ,\ j = 1, \ldots, n.$$

Recall $n = 2304$ and the noise is gaussian distributed. We use a zero mean gaussian distribution for simplicity.

Due to our usage of BPSK, σ^2 and the SNR $\dfrac{E_b}{N_0}$ share the following relation (note that since the code rate is 1/2, the message length $= m$)

$$mE_b = nE_s = \frac{\mathbb{E}\left[x_j^2\right]}{r} \tag{3.5}$$

$$\Rightarrow \frac{E_b}{N_0} = \frac{1}{2r\sigma^2} \tag{3.6}$$

$$\Rightarrow \sigma^2 = \frac{1}{2r} \cdot \frac{1}{E_b/N_0} \tag{3.7}$$

where the first equality is the input-output relation of the encoder for which the message of size m goes in, and encoded symbols of size n goes out into the channel. This is precisely the formula used to for providing the σ^2 parameter for the `Channel` and `Decoder` classes. Note $r = \frac{m}{n} = \frac{1}{2}$ is our code rate.

4. LDPC Decoding

4.1. Log Sum Product Algorithm

Due to the large nature of the **H** matrix for storing and handling in C++, we tend to efficient representions. For the implementation accompanying this report, 2 `.txt` files are created for storing the node node connectivity information (position of ones for each row) and vice versa. We utilize these `positions_cn_vn.txt` and `positions_vn_cn.txt` files in the encoding and decoding processes for parity check calculations and tanner graph traversing respectively. Therefore, simulation for each E_b/N_0 value keeps utilizes both files

Following the BI-AWGNC channel model, the received noisy symbol y_j is gaussian distributed. The decoder is initialized by setting all VN messages $L_{j\to i}$ equal to:

$$
\begin{aligned}
L_j = L(v_j \mid y_j) &= \log\left(\frac{\Pr(v_j = 0 \mid y_j)}{\Pr(v_j = 1 \mid y_j)}\right) = \log\left(\frac{\Pr(x_j = +1 \mid y_j)}{\Pr(x_j = -1 \mid y_j)}\right) \\
&= \log\left(\frac{\left(1+\exp\left(-\frac{2y_j}{\sigma^2}\right)\right)^{-1}}{\left(1+\exp\left(\frac{2y_j}{\sigma^2}\right)\right)^{-1}}\right) = \log\left(\frac{1+\exp\left(\frac{2y_j}{\sigma^2}\right)}{1+\exp\left(-\frac{2y_j}{\sigma^2}\right)}\right) \\
&= \log\left(\exp\left(\frac{2y_j}{\sigma^2}\right)\cdot\frac{1+\exp\left(\frac{2y_j}{\sigma^2}\right)}{1+\exp\left(\frac{2y_j}{\sigma^2}\right)}\right) = \frac{2y_j}{\sigma^2}
\end{aligned}
$$

according to the MAP estimation criterion.

It is critical to note that the LDPC code's **H** matrix can be regarded as a tanner (also bipartite) graphs, wherewith can apply belief propagation, sharing the soft information across check nodes and variable nodes (message passing). The received likelihood information becomes the message initialization for the variable nodes, to be sent to the check nodes.

To pass information between variable and check nodes, we create several arrays that keep track of the neighours of each node and its connectivity information (the number of edges connected to it). The neighbours refer to the (**H** matrix entries) connected to the specified check or variable node (row and column respectively). The other 2 2-d arrays, dubbed `L_ij` and `L_ji` of dimensions $m \times 10$ and $n \times 10$ are used for saving the message passed to the check nodes and variable nodes respectively. We note that the connectivity of each node does not exceed 10, in fact, it is often ≤ 7. Eitherway, we apply array indexing techniques to avoid needless indexing by utilizing the arrays storing the connectivity `cn_degrees` and `vn_degrees`. These node information are directly obtained from the accompanying `.txt` files which, which serve as a more compact representations of the parity check matrix.

The check node update is performed using the $\phi(\cdot)$ function and the α_{ji}, defined as follows:

$$
\begin{aligned}
L_{j\to i} &= \alpha_{ji}\beta_{ji}, \\
\alpha_{ji} &= \operatorname{sign}(L_{j\to i}), \\
\beta_{ji} &= |L_{j\to i}|,
\end{aligned}
$$

$$
\phi(x) = -\log[\tanh(x/2)] = \log\left(\frac{e^x+1}{e^x-1}\right), \quad x > 0 \tag{4.8}
$$

The above expressions are realized in the `phi_func()` function, which plays a critical role in the log SPA decoder.

Algorithm 1 The log Sum–Product Algorithm [1]

1: **Initialization:** For all j, initialize L_j according to the channel model. For all i, j such that $h_{ij} = 1$, set $L_{j\to i} = L_j = \dfrac{2y_j}{\sigma^2}$.

2: **CN update:** For each check node i and all $j \in N(i)$:

$$
L_{i\to j} = \prod_{j' \in N(i)\setminus\{j\}} \alpha_{j'i} \cdot \phi\left(\sum_{j' \in N(i)\setminus\{j\}} \phi(\beta_{j'i})\right),
$$

Transmit $L_{i\to j}$ to the variable nodes.

3: **VN update:** For each variable node j and all $i \in N(j)$:

$$
L_{j\to i} = L_j + \sum_{i' \in N(j)\setminus\{i\}} L_{i'\to j}
$$

Transmit $L_{j\to i}$ to the check nodes.

4: **LLR total:** For all $j = 0, \ldots, n-1$, compute:

$$
L_j^{\text{total}} = L_j + \sum_{i \in N(j)} L_{i\to j}
$$

5: **Stopping criteria:** For each $j = 0, \ldots, n-1$, set:

$$
\hat{v}_j = \begin{cases} 1 & \text{if } L_j^{\text{total}} < 0 \\ 0 & \text{otherwise} \end{cases}
$$

6: Compute $\mathbf{0} = \hat{\mathbf{c}}\mathbf{H}^\top$. If result is $\mathbf{0}$ or iteration limit $= 20$ is reached, **stop**; else, go to Step 2.

These functions come about due to the parity check requirement on the check nodes. Each check node acts as a Single-Parity-Check code, from which we obtain the the CN update equation that is simplified to the one in algorithm 1 [1, 3].

The VN update follows a follows a simply message gathering strategy from the check nodes. This is because it collects messages relevant to detection bits are assumed to be independent, hence, the only goal for the VN update round is to aggregate messages relevant for correct hard-decision decoding. This is similarly realized within our `Decoder`

class, first aggregating information from all neighbours and then updating each `L_ji` value after subtracting the pertaining `L_ij`.

Before hard-decision making, the total LLR from all neighbours to the variable node is aggregated and then hard-decisions are made according to the modulation mapping and likelihood message information.

Finally, it is critical to verify if the decoded codeword is valid. as shown in step 6, otherwise, the procedure continues from step 2 until maximum number of iterations 20 is reached for each codeword.

4.2. Minsum Algorithm

In this section, a basic minsum algorithm is implemented. The motivation here is to show that an approximation of the above log SPA algorithm often leads to faster but less accurate decoders, although the performance must be acceptable for all practical purposes. The goal is to speed up the single-parity-check evaluation of messages. We only consider the simple approximation case, stemming from the properties of the $\phi(\beta_{j'i})$ function. Therefore, the only difference between the minsum approximation decoder and the log SPA is the

$$\textbf{CN update: } L_{i\to j} = \prod_{j'\in N(i)\setminus\{j\}} \alpha_{j'i} \cdot \min_{j'\in N(i)\setminus\{j\}} \beta_{j'i}. \tag{4.9}$$

The above CN approximation can be proven by considering the properties of the $\phi(\cdot)$ function defined in 4.8. First, it is useful to visualize the function as follows:

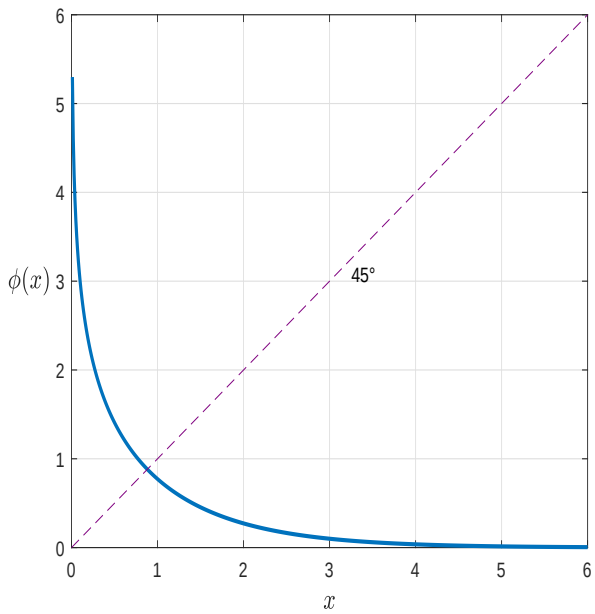

Figure 4.1.: $\phi(x)$ function

Observing the nature of the $\phi(x)$ function, we can observe that it diminishes exponentially, is symmetric and self-inverse [1]. As a result, we can estimate

$$\phi(x_1)+\phi(x_2)+\ldots+\phi(x_n) \simeq \phi(\max_{i=1,\ldots,n} x_i).$$

Consequently, we can also apply the self-inverse property and our CN update equation can be modified as

$$\phi\left(\sum_{j'} \phi(\beta_{j'i})\right) \simeq \phi\left(\phi\left(\min_{j'} \beta_{j'i}\right)\right) = \min_{j'\in N(i)\setminus\{j\}} \beta_{j'i}.$$

This directly leads us to the updated CN update provided in equation 4.9, alongside sign evaluation.

We can observe minsum significantly cuts down the computation involved with `phi_func()`, reducing the complexity of the check node update in the SPA decoder from $\mathcal{O}(m \cdot d_c^2)$ to $\mathcal{O}(m \cdot d_c)$, moreso without factoring the non-linear computations for the `exp()` and `log`

functions in C++. Another advantage minsum offers is that there is no need for the decoder to utilize the channel noise variance, σ^2. This is because this estimated positive constant will have no effect on the sign nor on the minimum value selected. Hence, minsum eliminates the need for channel estimation in the decoding process. In our implementation, we utilize sigma anyway, but the results are unchanged when it is removed. Everything else about the simulation stays the same as for the log SPA implementation.

5. Results and Discussion

5.1. BER Performance

We expect our implemented decoder to perform better decoding as the SNR increases. For each SNR value, we randomly generate a large number of message bits, encode them into codewords, transmit over a BI-AWGNC and then decode using the pertaining algorithms. The message part of the decoded codeword is extracted and the total number of bit errors is calculated as follows:

$$\text{error rate} = \frac{\text{sum of total errors}}{\text{accumulated message size}}$$

For each SNR, the same process is repeated over multiple rounds and the total message errors are accumulated, alongside the total number of bits transmitted, until convergence is achieved using a 95% interval for the moving average BER. These are realized within the `BERSimulator` and `StatisticsTracker` classes. Both serve as important helper functions for the overall simulation.

Having obtained the BER values at convergence for each simulated SNR (EbN0) value, we can observe the waterfall trend of the BER curve. The number of simulation errors, and convergence rate of the code significantly increases with increasing BER, achieving up to $\sim 10^{-7}$ for 2.2 dB.

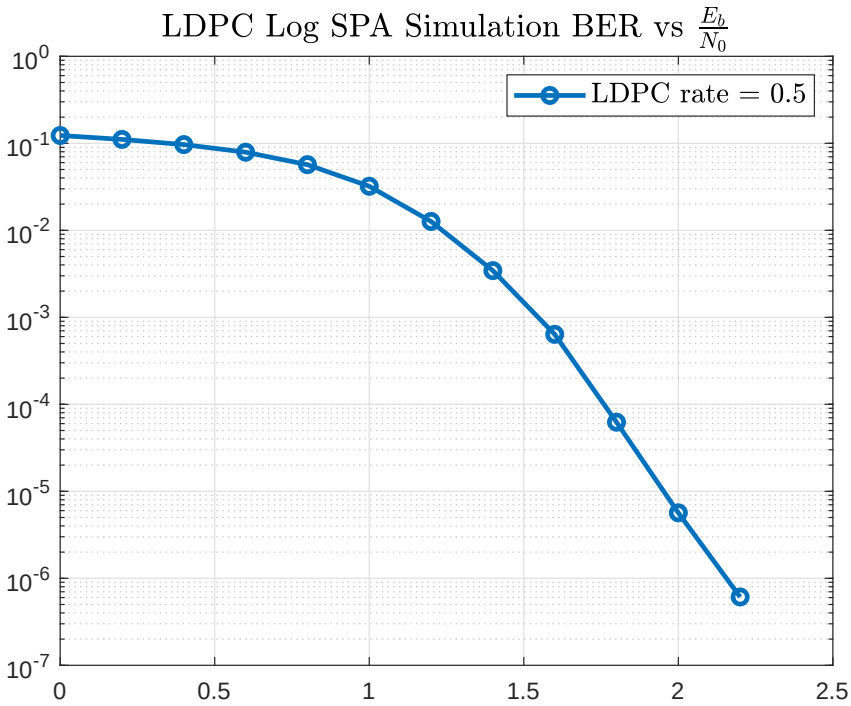

Figure 5.1.: Log SPA BER Performance Curve

It is important to note that for the high SNR value 2.2 dB, convergence is hard to achieve due to very few errors encountered. As such, we add a relaxed convergence constraint such that beyond $\sim 10^7$ bits simulated, the current BER becomes the final. In other experiments, we also increase the number of errors required for relaxed convergence to 100 (reasonable number), and run the 2.2 dB point alone, to obtain a more credible value. We settle on the corresponding BER value in the table herein, as appropriate.

In reality, there are 2^{1152} valid codewords. This is an incredible number which is virtually impossible to iteratively simulate through. Monte carlo methods as applied in this project, therefore provide an appropriate approach to reasonably estimate the BER points.

5.2. BER Performance of Minsum vs Log SPA

The minsum vs Log SPA figure follows a similar trend to 5.1. The minsum decoder shows a degraded BER performance since it is an approximation of log SPA. It is however important to note that this can be improved using other versions of minsum approximation such as attenuated and offset minsum decoders. In addition, we can observe that the minsum decoder is approximately an order of magnitude above the log SPA counterpart, up to approximately 1 dB worse.

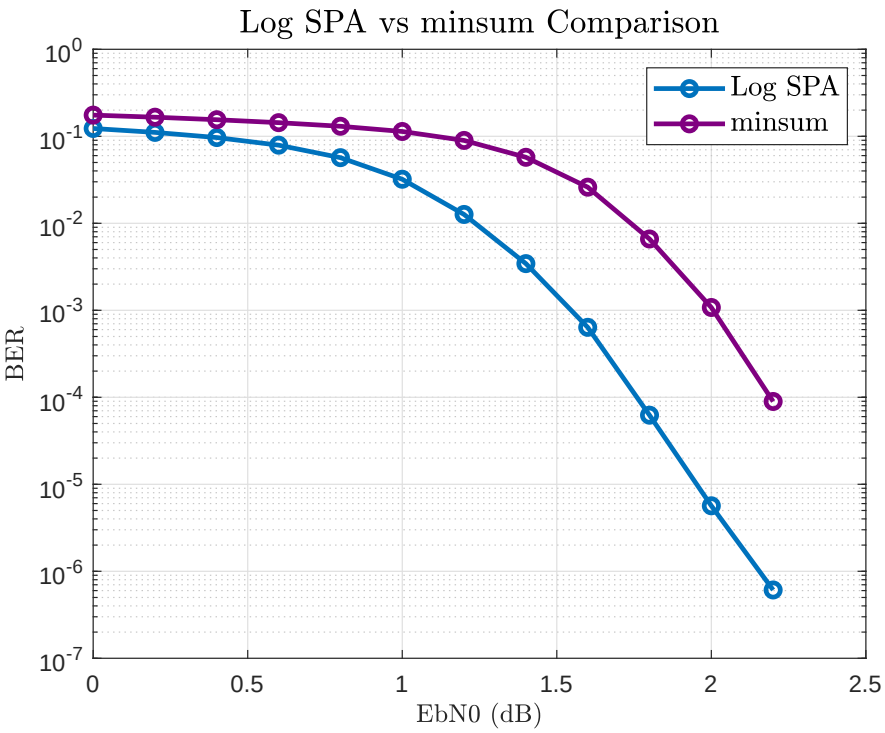

Figure 5.2.: Log SPA vs minsum BER Curve

The following table 5.1 displays the BER values of both simulations, with more details including number of errors and number of bits simulated until converged value, provided in the excel sheet `simulation_results.xlx`.

Table 5.1.: BER Results Log SPA and Min-Sum Algorithms

$\mathbf{E_b/N_0}$ **(dB)**	**BER: Log SPA**	**BER: minsum**
0.0	1.230e-01	1.752e-01
0.2	1.109e-01	1.658e-01
0.4	9.672e-02	1.551e-01
0.6	7.907e-02	1.435e-01
0.8	5.682e-02	1.304e-01
1.0	3.209e-02	1.136e-01
1.2	1.264e-02	8.966e-02
1.4	3.440e-03	5.737e-02
1.6	6.371e-04	2.602e-02
1.8	6.226e-05	6.601e-03
2.0	5.660e-06	1.077e-03
2.2	6.097e-07	8.950e-05

5.3. Complexity analysis

Here, we provide complexity analysis of the encoder and decoder algorithms implemented. We also compare the implementation complexity of the minsum algorithm as specified in the section 4.2 with log SPA. We find that minsum indeed runs faster due to the reduced complexity. First,

1. **Encoder**. The encoder is precisely the same for both the log SPA and minsum algorithm. Our implemented algorithm takes $\mathcal{O}(m \cdot d_c) + \mathcal{O}(m) + \mathcal{O}(1056 \cdot d_c)$, corresponding to syndrome calculation, first parity block calculation (96 bits) and remaining parity bits calculation respectively.

2. **Decoder** For the log SPA decoder, our implementation takes $\mathcal{O}(m \cdot d_c^2)$ for the CN update, $\mathcal{O}(n \cdot d_v^2)$ for the VN update, $\mathcal{O}(n \cdot d_v^2)$ for the total LLR and $\mathcal{O}(m \cdot d_c)$ for the stopping criterion.

3. **General Simulation Strategy**

   For simulation, we apply monte carlo methods to continue calculating the BER until a convergence criterion using confidence intervals is achieved. We use a 95% confidence interval. This overall slows down the simulation and results in several hours of waiting, however, it helps in the estimation of a somewhat true BER for any given E_b/N_0.

6. Conclusion and Future Work

In this project, we have implemented the log SPA algorithm, a belief-propagation-based soft messaging passing algorithm for decoding a 1152×2304 QC-LDPC code, commonly used in the WiMaX standard. We further modified the decoder to the simple minsum decoder case. As a result, we show that while the minsum decoder has a lower complexity, it is an approximation of the original log SPA algorithm and as a result, it's BER performance degrades.

For future work, we will consider modifying the `Channel` class to model a BSC channel and a BEC. Consequently, the decoders will be modified to implement the bit-flipping (Gallager's algorithm A) and iterative filling algorithms respectively. We will also consider other mofications to the minsum decoder such as the attenuated and off-set decoders, and other state of the art techniques. This will further build capacity and comfort with working on similar projects in the future. Further studies may also be performed on the construction procedure for irregular LDPC codes, which often entail solving linear optimization problems (relaxed), utilizing the node degree distribution polynomials.

In conclusion, LDPC codes provide incredible performance, achieving very low bit error rates as SNR increases. This are also long codes, making them ground-breaking and suitable for several high speed applications including the recent 5G NR LDPC codes.

A. Monte Carlo Methods and Useful Statistics

Here, we present the statistical methods and convergence criteria utilized to obtain the BER values in our LDPC simulation. The Monte Carlo simulation employs key parameters to ensure statistical reliability: $1 \times 10^7 \rightarrow 2 \times 10^7$ minimum information bits per SNR point, $100 \rightarrow 200$ minimum errors before convergence assessment, and a maximum of $150,000$ simulation rounds per SNR point. Convergence is assessed every 50 rounds using a statistical window of 100 recent BER observations.

Statistical Measures and Convergence Detection

For the sliding window of $n = 100$, recent BER observations $x_1, x_2, \ldots, x_n$, the sample mean is calculated as

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i.$$

The implementation maintains a circular buffer (vector) where the oldest BER value is replaced with the most recent observation every 50 rounds. The sample variance is also obtained using

$$s^2 = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})^2,$$

with the sample standard deviation following as $s = \sqrt{s^2}$.

For 95% confidence interval estimation, we assume the BER samples follow an approximately normal distribution. The confidence interval for the true mean μ_{BER} is constructed as:

$$\mu_{\text{BER}} \in \left[\bar{x} - z \cdot \frac{s}{\sqrt{n}}, \bar{x} + z \cdot \frac{s}{\sqrt{n}} \right]$$

where $z = 1.96$ is the critical value for 95% confidence from the standard normal distribution.

Statistical convergence is declared when the relative half-width of the 95% confidence interval falls below a specified threshold [4, 5]:

$$\frac{1.96 \cdot s/\sqrt{n}}{\mu_{\text{BER}}} < 0.1,$$

ensuring the confidence interval half-width is within 10% of the estimated mean BER [4, 5]. Following this, we implement a dual stopping criteria for the BER estimation: primary convergence requires both minimum bits $\sim 10^7$ and errors (200) to be satisfied

along with statistical convergence, while a fallback criterion relaxes the error threshold to 50 errors for high SNR scenarios with extended bit counts ($\sim 10^8$ bits).

The simulation uses a fixed seed (42) for reproducibility and generates independent Gaussian noise samples using the `Normal()` function for proper AWGN channel modeling, alongside the calculated channel `sigma` value from section 3.1. Progress monitoring occurs every 200 rounds, displaying current round number, accumulated errors, total bits simulated, and current BER estimate. This statistical framework ensures that reported BER values represent statistically significant estimates with quantified confidence bounds, particularly important for the low BER regime, typical of LDPC codes at moderate to high SNR values.

Bibliography

[1] Ryan, William, and Shu Lin. Channel codes: classical and modern. Cambridge university press, 2009.

[2] Chia-Yu Lin, Chih-Chun Wei and Mong-Kai Ku, "Efficient encoding for dual-diagonal structured LDPC codes based on parity bit prediction and correction," APCCAS 2008 - 2008 IEEE Asia Pacific Conference on Circuits and Systems, Macao, China, 2008, pp. 1648-1651, doi: 10.1109/APCCAS.2008.4746353.

[3] Hasani, Alireza, Lopacinski, Lukasz, Kraemer, Rolf. (2021). Reduced-complexity decoding implementation of QC-LDPC codes with modified shuffling. EURASIP Journal on Wireless Communications and Networking. 2021. 183. 10.1186/s13638-021-02056-5.

[4] DATAtab: DATAtab Team (2025). DATAtab: Online Statistics Calculator. DATAtab e.U. Graz, Austria. URL `https://datatab.net`

[5] Youtube: Learning OnDemand, (2020), Hypothesis Testing - Hypothesis Testing - Making Decision Using Confidence Interval, URL `https://www.youtube.com/watch?v=yuIvN5ETtcI`.

[6] RPTU, "Channel Codes Database, WiMax LDPC", [Online]. Available: `https://rptu.de/en/channel-codes/channel-codes-database/wimax-ldpc`.